

Exploratory Data Analysis and Visualisation

Dr Hadeel Saadany

Hadeel.Saadany@bcu.ac.uk



What is Exploratory Data Analysis



BIRMINGHAM CITY
University

EDA is an approach for data analysis using variety of techniques to gain insights about the data.

Basic steps in any exploratory data analysis:

- Summarising Data
- Statistical Analysis
- Visualization for trend analysis, anomaly detection, outlier detection (and removal).

Importance of Visualisation



Improve understanding of variables by extracting averages, mean, minimum, and maximum values, etc.



Discover errors, outliers, and missing values in the data.



Identify patterns by visualizing data in graphs such as bar graphs, scatter plots, heatmaps and histograms.

Steps in EDA

1. Problem Definition

- Define the main objective of the analysis and what is the dataset about.

2. Data Preparation

- Clean data, delete non-relevant features, transform data, etc.

3. Data Analysis

- Descriptive statistics through graphs, correlations, etc.

4. Representation of Results

- Presenting your dataset to the target audience through graphs

Hands-on

- Download the Pokemon dataset from:

<https://www.kaggle.com/datasets/rounakbanik/pokemon>

- Unzip the folder, and save the data file in a location you'll remember.



Reading in the data

- First we import the Python packages we are going to use.
- Then we use Pandas to load in the dataset as a data frame.

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
import seaborn as sns
df1 = pd.read_csv('C:/Users/lb690/Google Drive/Teaching/Q-Step_workshops_2019_2020/pokemon_dataset.csv', index_col=0, encoding = "ISO-8859-1")
```

NOTE: The argument `index_col` argument states that we'll treat the first column of the dataset as the ID column.

NOTE: The encoding argument allows us to bypass an input error created by special characters in the data set.

Step 1: Understand your data

Examine the data set



BIRMINGHAM CITY
University

```
print(df1.head())
```

	Name	Type 1	Type 2	Total	...	Sp. Def	Speed	Stage	Legendary
#					...				
1	Bulbasaur	Grass	Poison	318	...	65	45	1	False
2	Ivysaur	Grass	Poison	405	...	80	60	2	False
3	Venusaur	Grass	Poison	525	...	100	80	3	False
4	Charmander	Fire	NaN	309	...	50	65	1	False
5	Charmeleon	Fire	NaN	405	...	65	80	2	False

```
print(df1.describe())
```

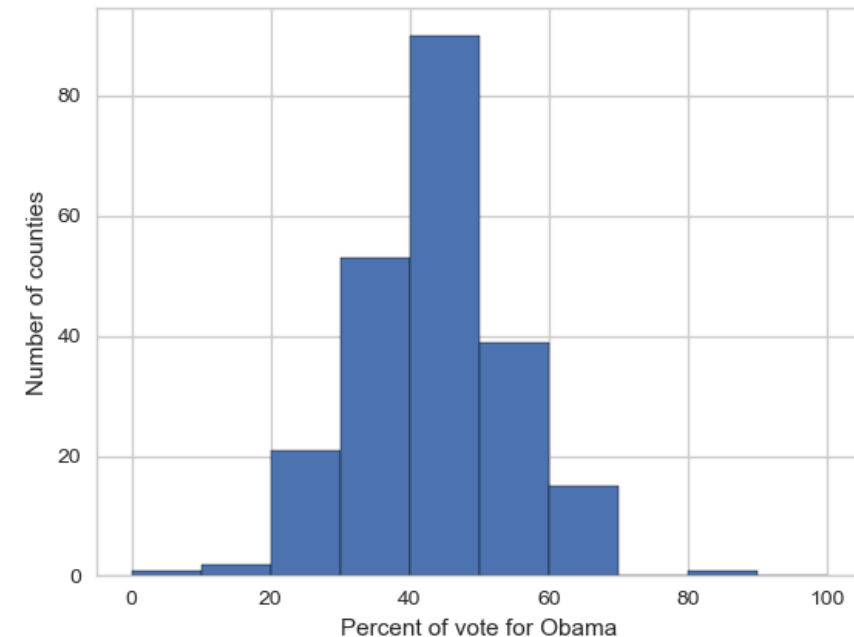
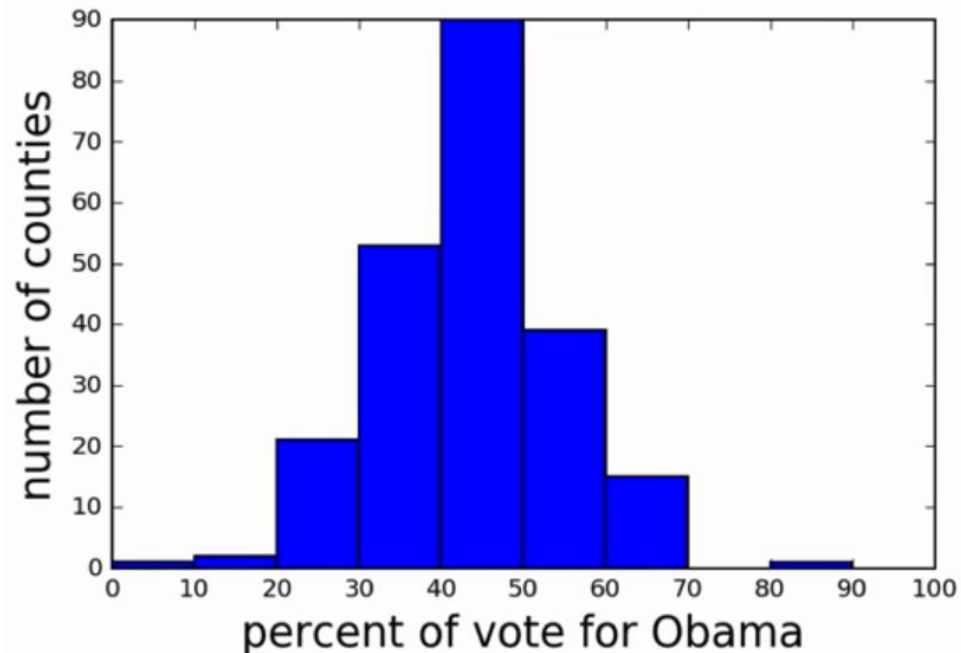
	Total	HP	Attack	...	Sp. Def	Speed	Stage
count	151.00000	151.000000	151.000000	...	151.000000	151.000000	151.000000
mean	407.07947	64.211921	72.549669	...	66.019868	68.933775	1.582781
std	99.74384	28.590117	26.596162	...	24.197926	26.746880	0.676832
min	195.00000	10.000000	5.000000	...	20.000000	15.000000	1.000000
25%	320.00000	45.000000	51.000000	...	49.000000	46.500000	1.000000
50%	405.00000	60.000000	70.000000	...	65.000000	70.000000	1.000000
75%	490.00000	80.000000	90.000000	...	80.000000	90.000000	2.000000
max	680.00000	250.000000	134.000000	...	125.000000	140.000000	3.000000

Step 2 EDA through Data Visualisation

Libraries for data visualisation

Seaborn and Matplotlib

- Matplotlib is a powerful, but sometimes unwieldy, Python library.
- Seaborn provides a high-level interface to Matplotlib and makes it easier to produce graphs like the one on the right.
- Some IDEs incorporate elements of this “under the hood” nowadays.



Visualising numerical data

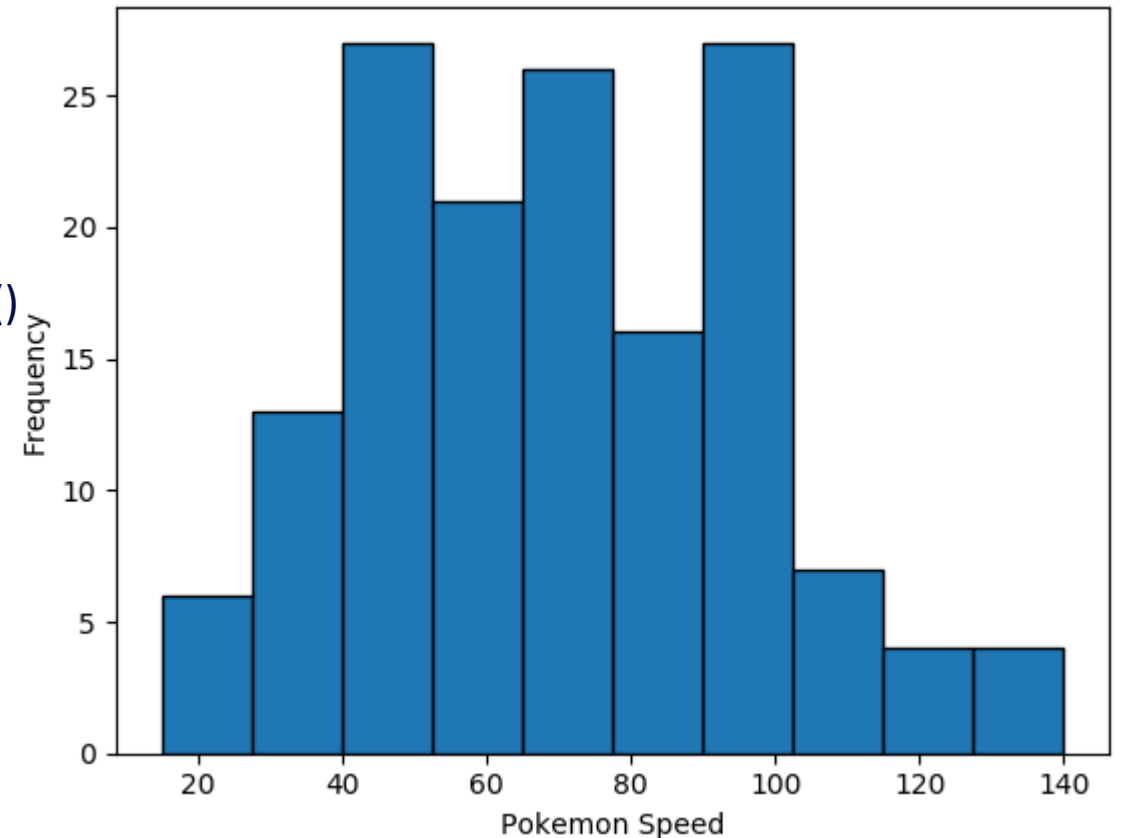
Histograms

Histogram plots are used to depict the distribution of any **continuous variable**.

```
g = plt.hist(df1['Speed'], histtype='bar', ec='black',)
g = plt.xlabel('Pokemon Speed')
g = plt.ylabel('Frequency')
plt.show()
```

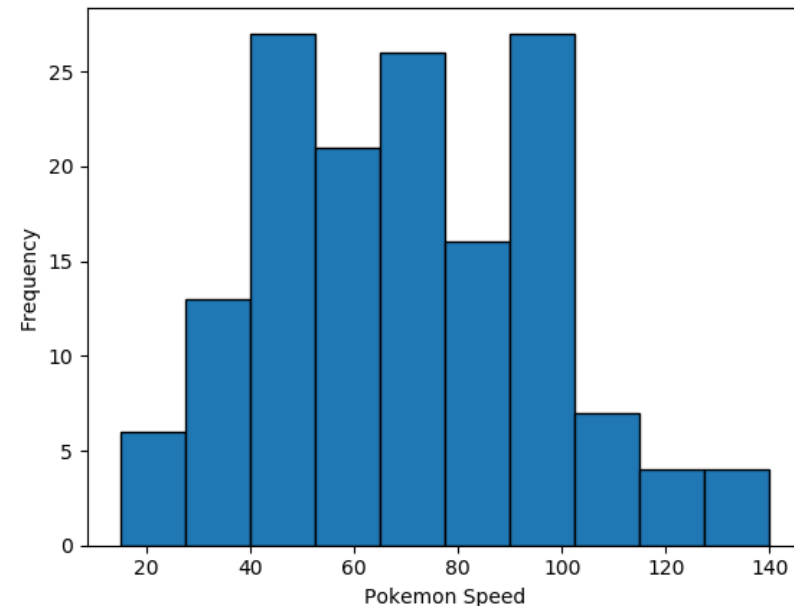
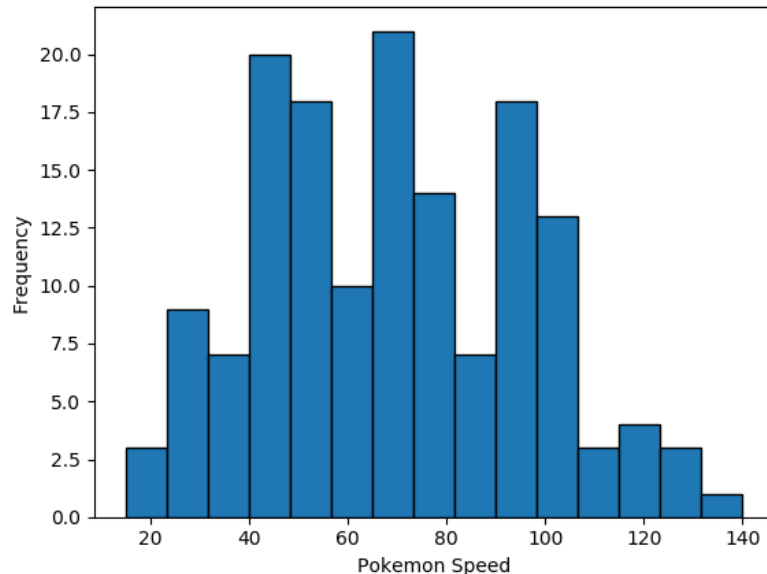
Saving Figures to File (use this line of code before the plt.show())

```
In [5]: fig.savefig('my_figure.png')
```



Bins

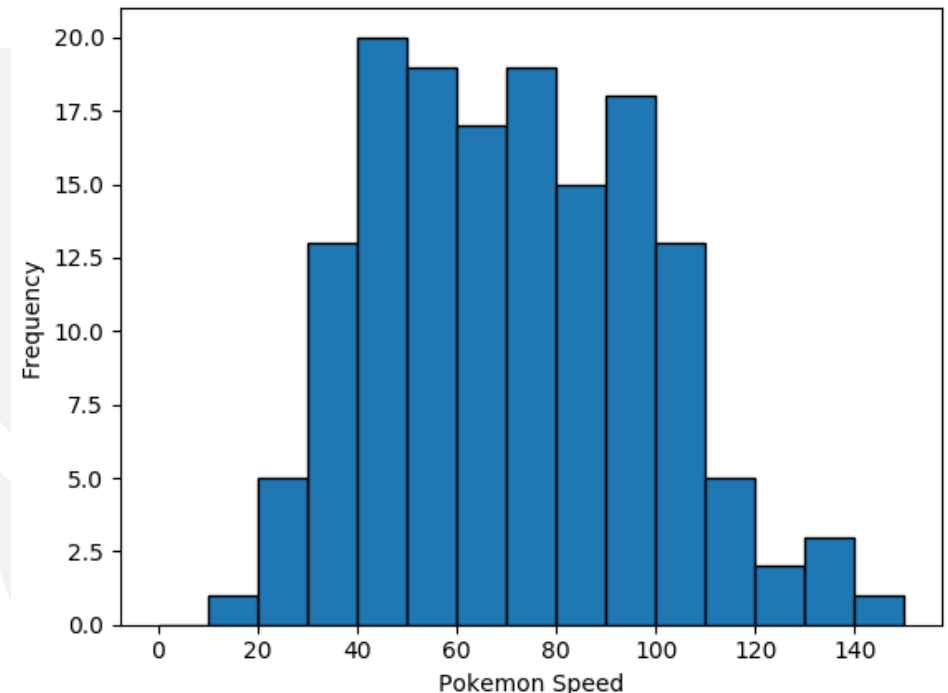
- You may have noticed the two histograms we've seen so far look different, despite using the **exact** same data.
- This is because they have different bin values.
- The left graph used the default bins generated by `plt.hist()`, while the one on the right used bins that I specified.





- There are a couple of ways to manipulate bins in matplotlib.
- Here, I specified where the edges of the bars of the histogram are; the bin edges.

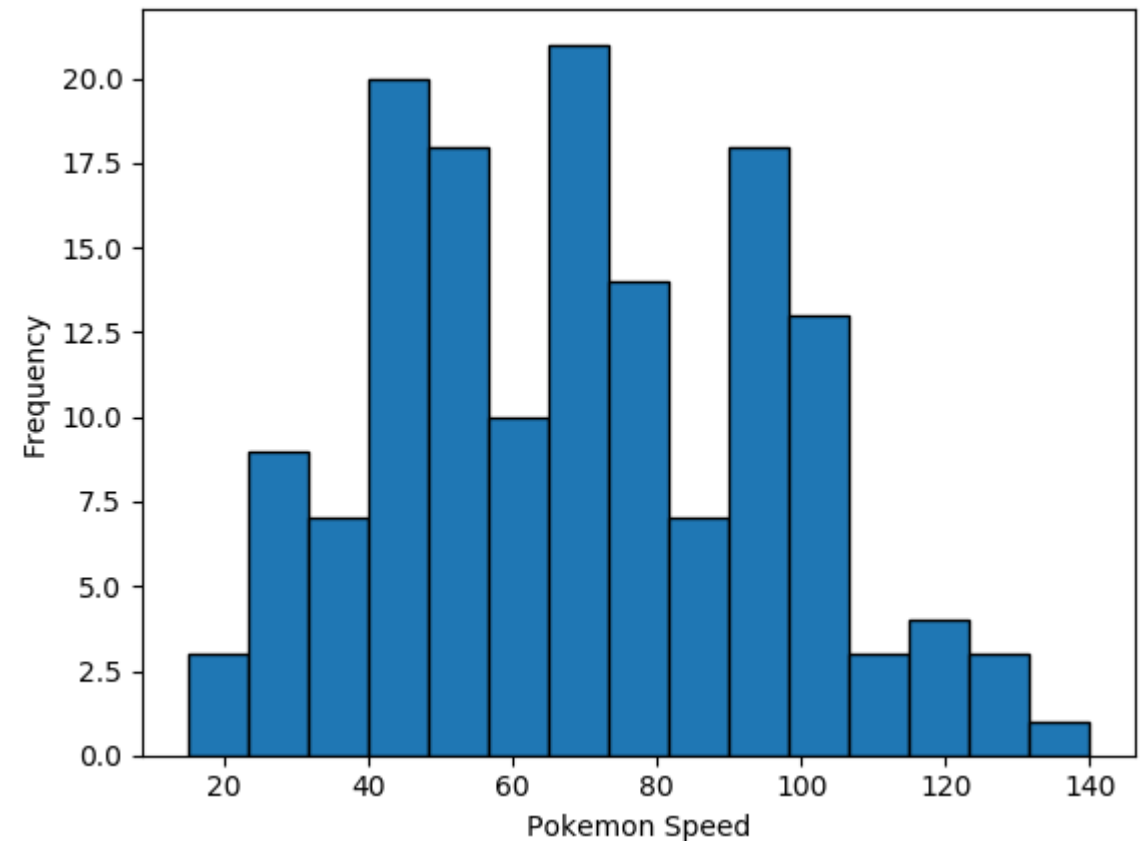
```
bin_edges = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150]  
g = plt.hist(df1['Speed'], histtype='bar', ec='black', bins=bin_edges)  
g = plt.xlabel('Pokemon Speed')  
g = plt.ylabel('Frequency')  
plt.show()
```





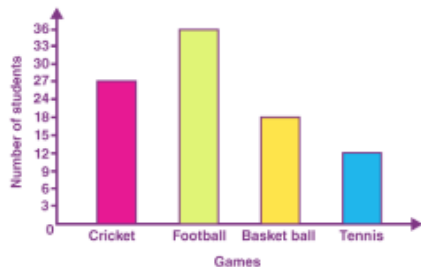
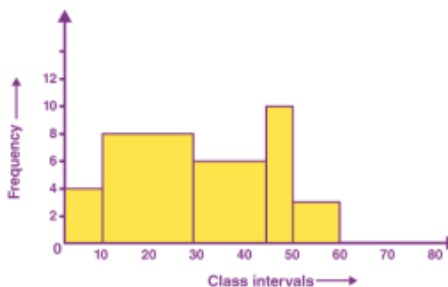
- You could also specify the number of bins, and Matplotlib will automatically generate a number of evenly spaced bins.

```
g = plt.hist(df1['Speed'], histtype='bar', ec='black', bins=15)  
g = plt.xlabel('Pokemon Speed')  
g = plt.ylabel('Frequency')  
plt.show()
```





What are the Differences Between Bar Chart and Histogram?

Bar graph	Histogram
The bar graph is the graphical representation of categorical data.	A histogram is the graphical representation of quantitative data.
There is equal space between each pair of consecutive bars.	There is no space between the consecutive bars.
The height of the bars shows the frequency, and the width of the bars are same.	The area of rectangular bars shows the frequency of the data and the width of the bars need not to be same.
 BYJU'S The Learning App	 BYJU'S The Learning App

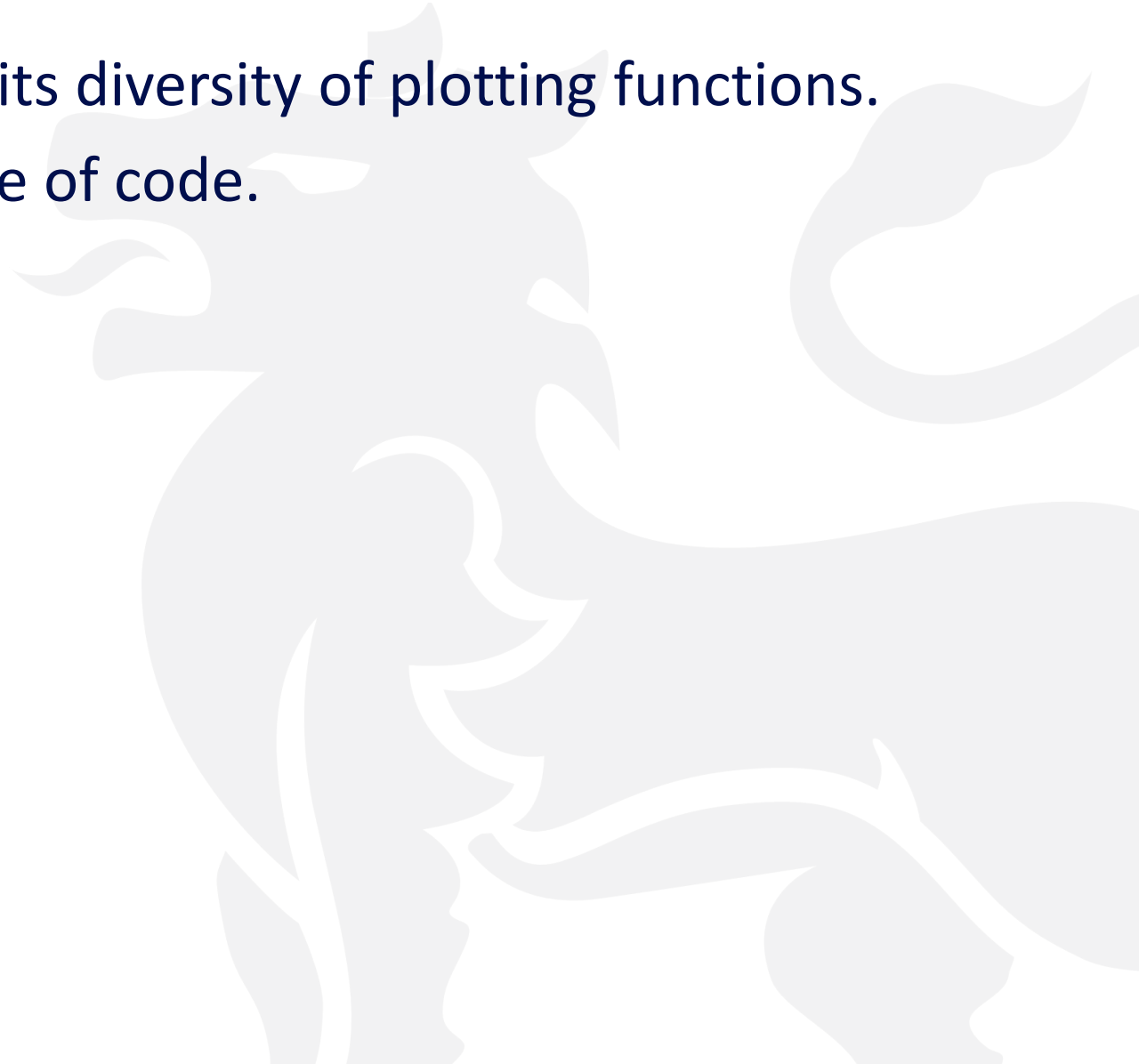


Benefits of Seaborn

- Seaborn offers:
 - Using default themes that are aesthetically pleasing.
 - Setting custom colour palettes.
 - Making attractive statistical plots.
 - Easily and flexibly displaying distributions.
 - Visualising information from matrices and DataFrames.
- The last three points have led to Seaborn becoming the exploratory data analysis tool of choice for many Python users.

Plotting with Seaborn

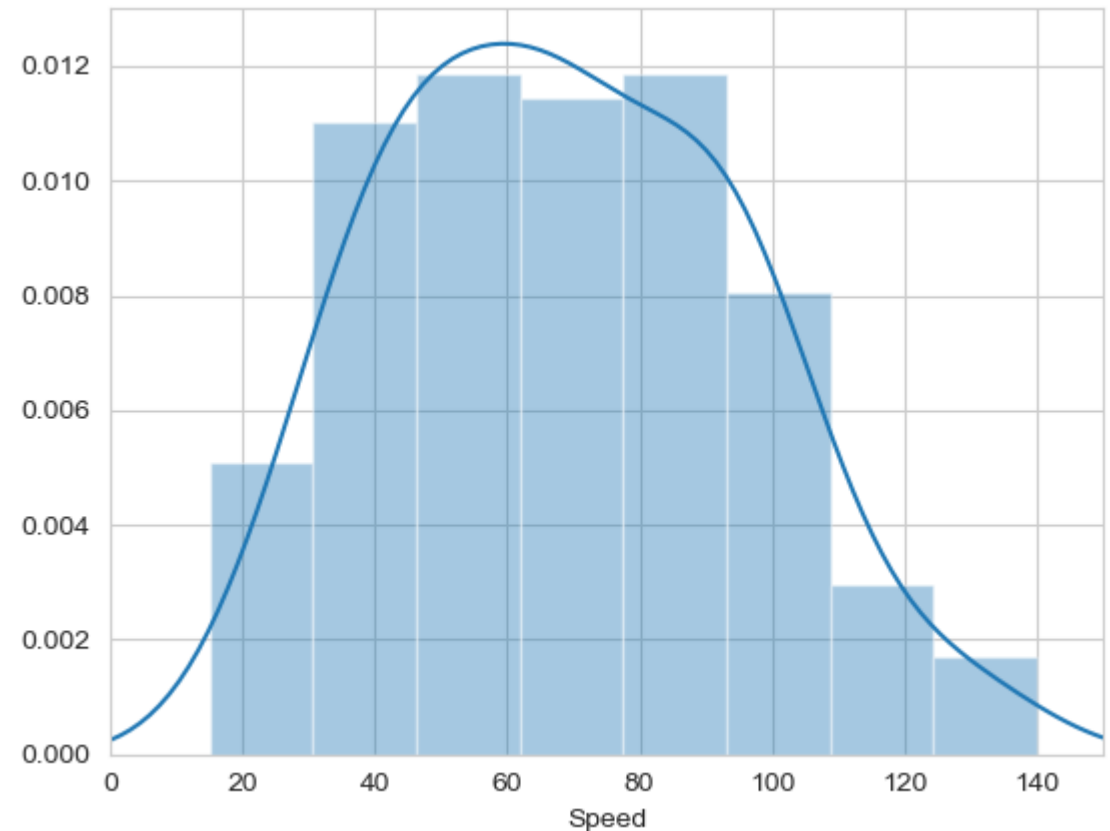
- One of Seaborn's greatest strengths is its diversity of plotting functions.
- Most plots can be created with one line of code.
- For example....



Histograms

- Allow you to plot the distributions of numeric variables.

```
sns.set_style()  
sns.distplot(df1.Speed)
```



Other types of graphs: Creating a scatter plot



BIRMINGHAM CITY
University

Seaborn “linear model plot”
function for
creating a scatter
graph

↑

```
sns.lmplot(x='Attack', y='Defense', data=df1)
```

↑

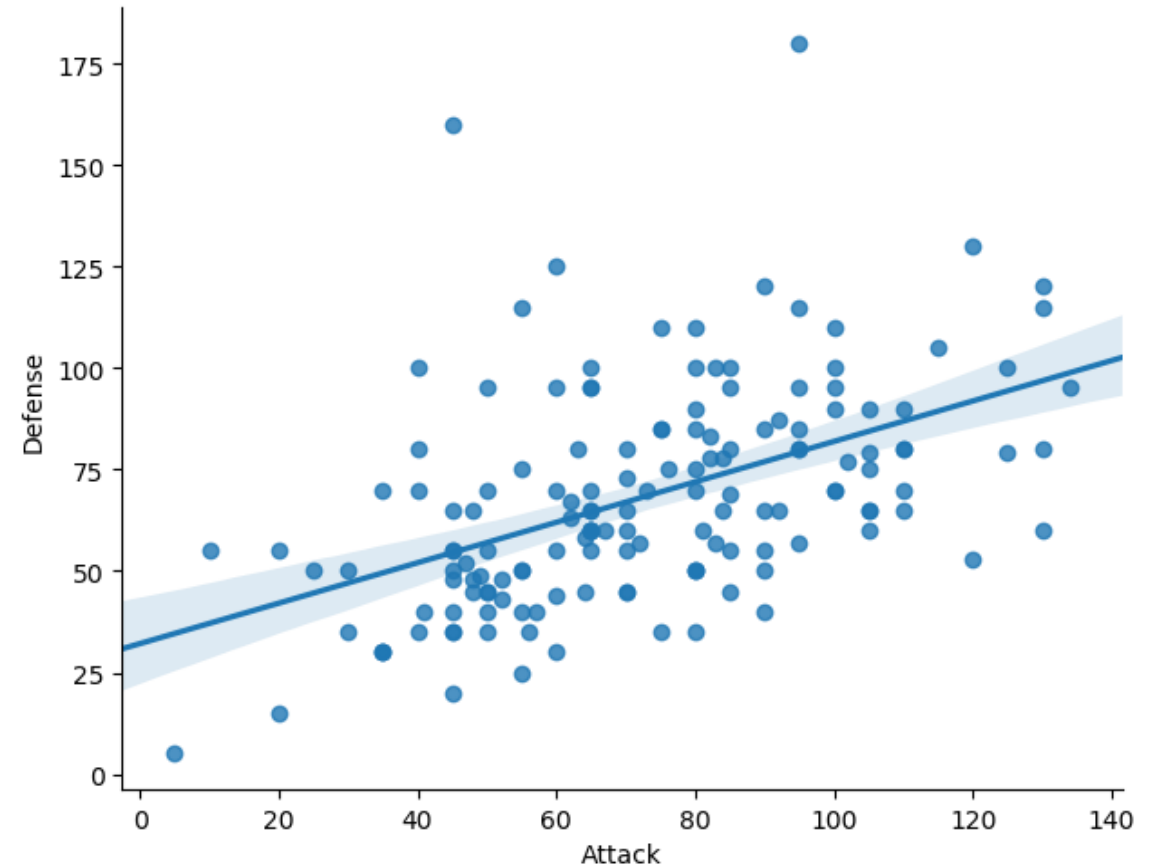
Name of variable we
want on the x-axis

↑

Name of variable we
want on the y-axis

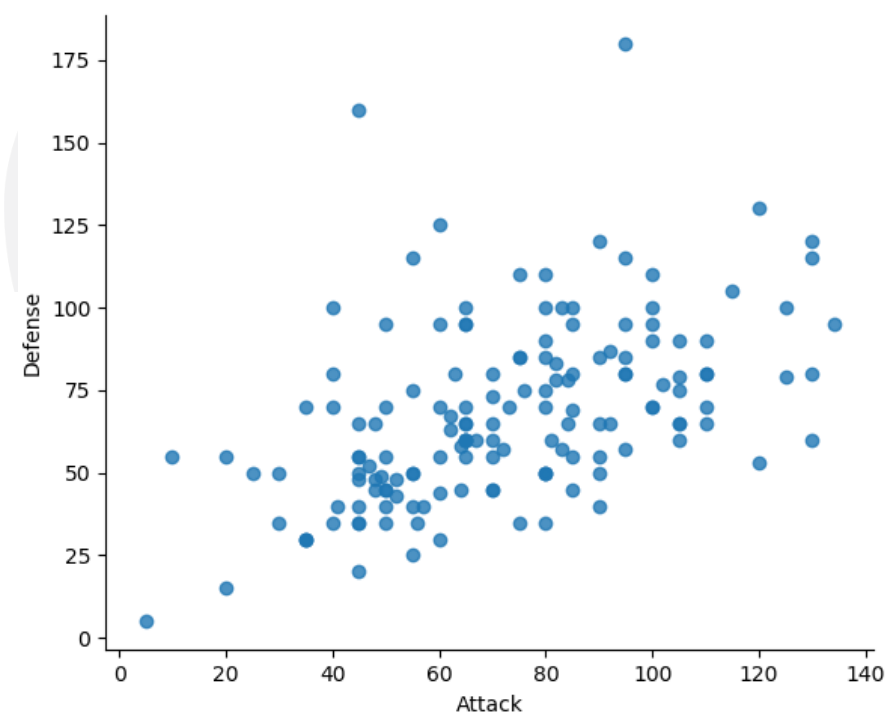
↑

Name of our
dataframe fed to the
“data=” command



- Seaborn doesn't have a dedicated scatter plot function.
- We used Seaborn's function for fitting and plotting a regression line; hence `lmplot()`
- However, Seaborn makes it easy to alter plots.
- To remove the regression line, we use the `fit_reg=False` command

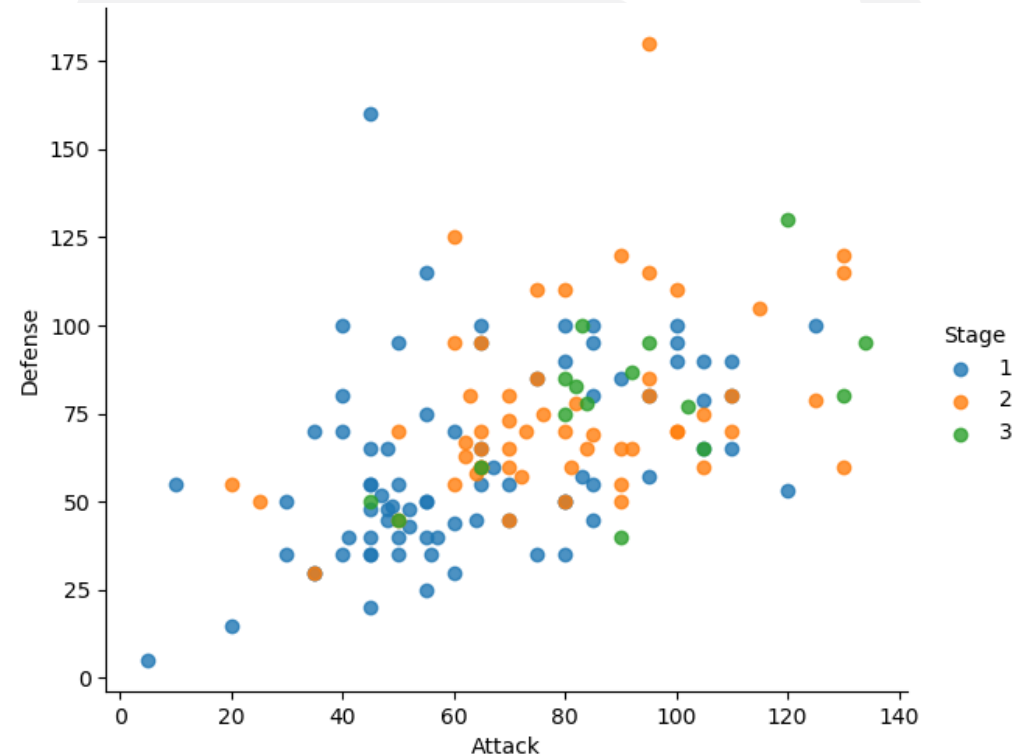
```
sns.lmplot(x='Attack', y='Defense', data=df1, fit_reg=False)
```



The hue function

- Another useful function in Seaborn is the `hue` function, which enables us to use a variable to colour code our data points.

```
sns.lmplot(x='Attack', y='Defense', data=df1,  
           fit_reg=False,  
           hue='Stage')
```



Factor plots

- Make it easy to separate plots by categorical classes.

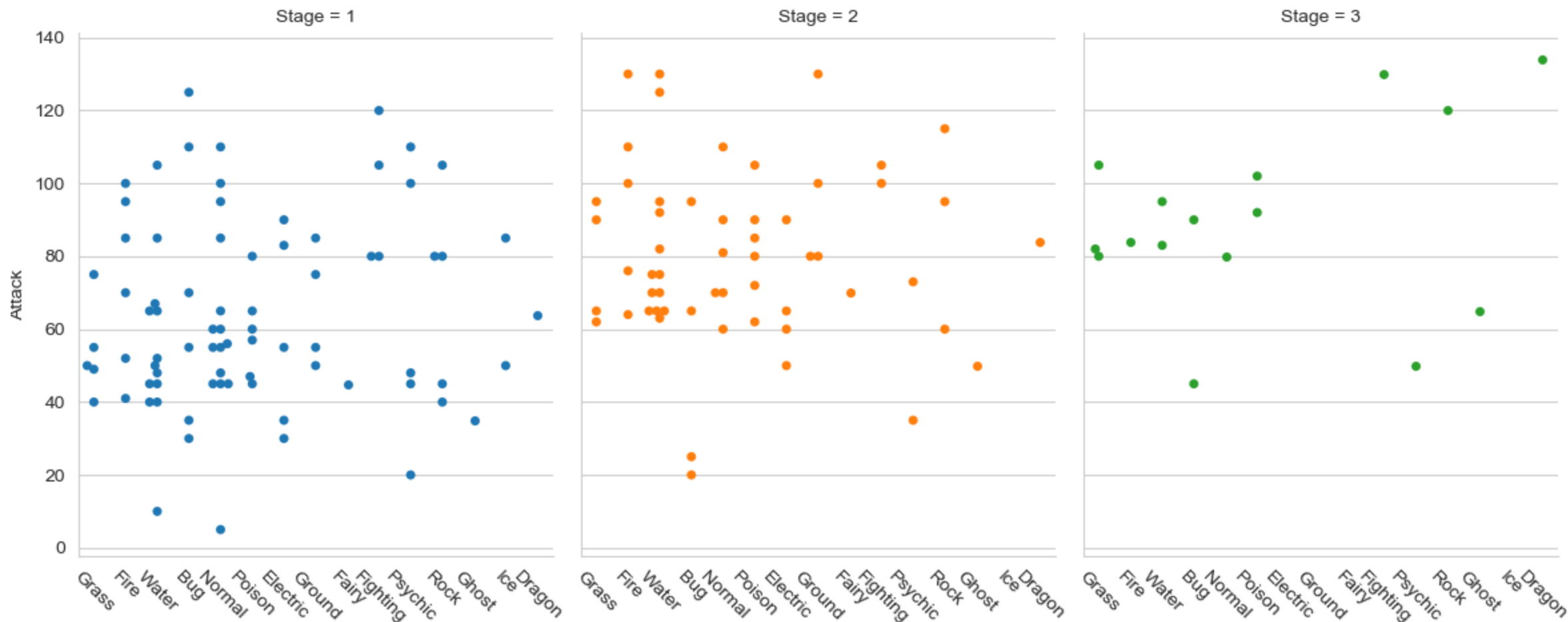
```
g = sns.factorplot(x='Type 1',  
                  y='Attack',  
                  data=df1,  
                  hue='Stage',  
                  col='Stage',  
                  kind='swarm')  
g.set_xticklabels(rotation=-45)
```

Colour by stage.

Separate by stage.

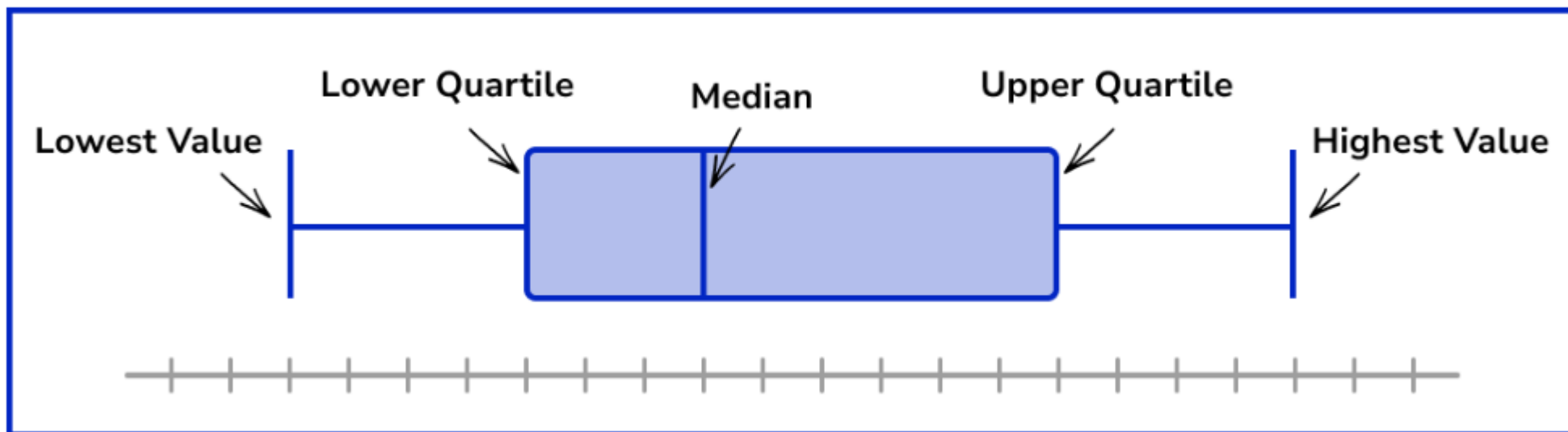
Generate using a swarmplot.

Rotate axis on x-ticks by 45 degrees.



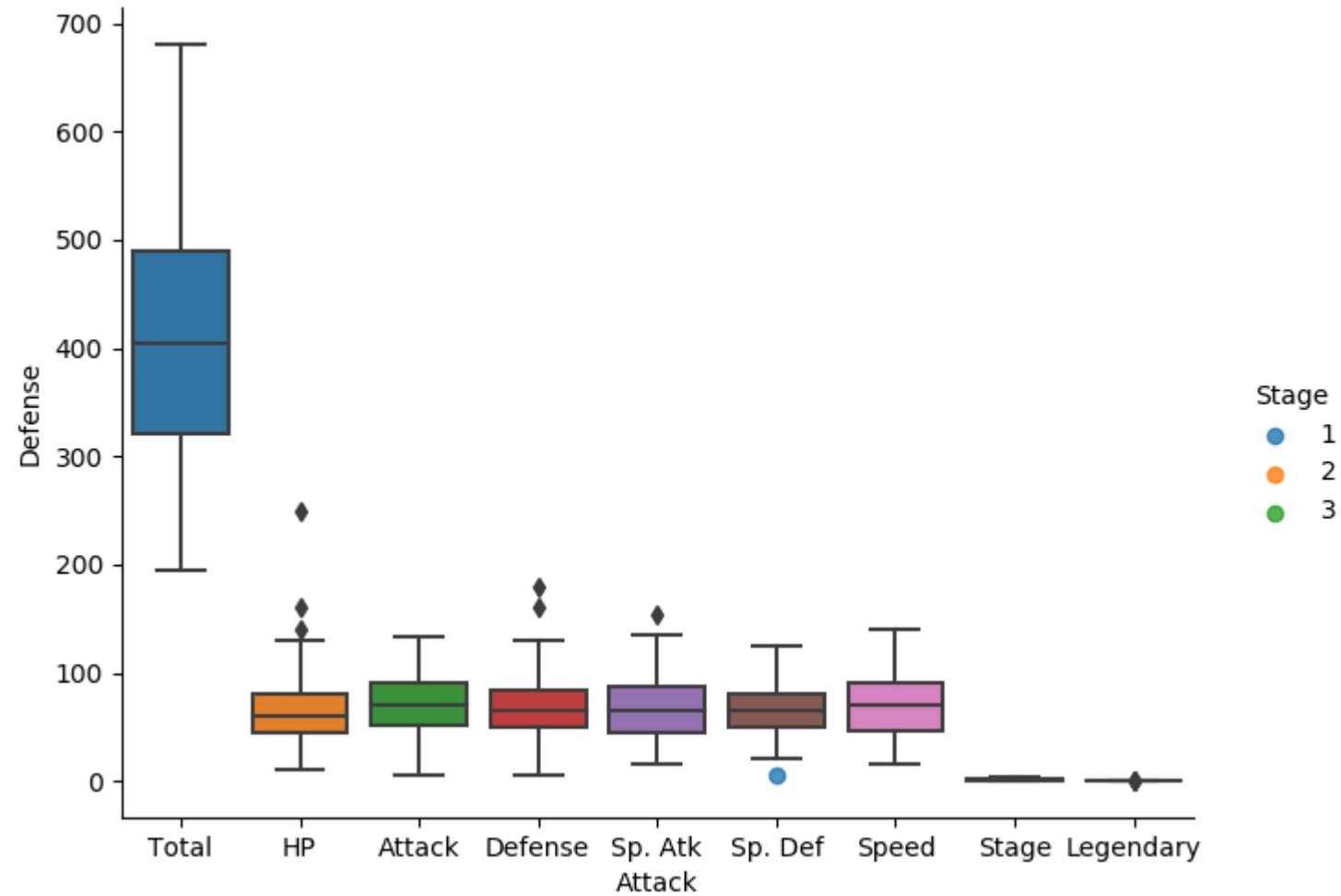
Box plots

- Box plots are particularly useful for data analysis when comparing two or more data sets; it is easy to make visual comparisons of average (median) and spread (range and interquartile range).
- When data is skewed (i.e. the distribution of data is not symmetrical or near-symmetrical), or there are many outliers or extreme values, a box plot provides better data visualisation than other chart types or graphs.



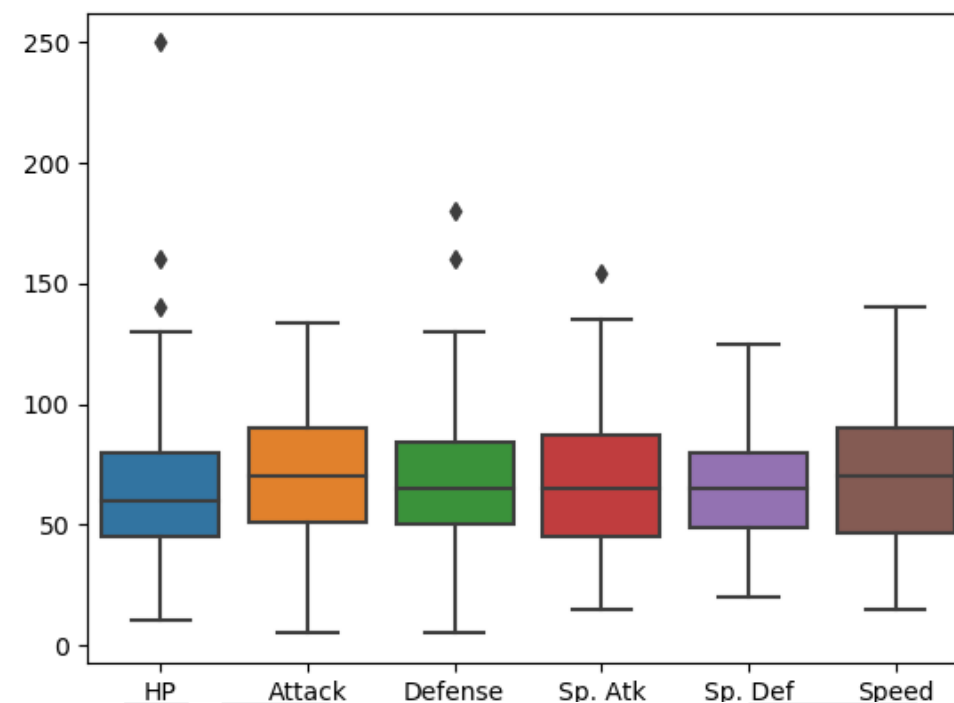
A box plot

```
sns.boxplot(data=df1)
```



- The total, stage, and legendary entries are not combat stats so we should remove them.
- Pandas makes this easy to do, we just create a new dataframe
- We just use Pandas' `.drop()` function to create a dataframe that doesn't include the variables we don't want.

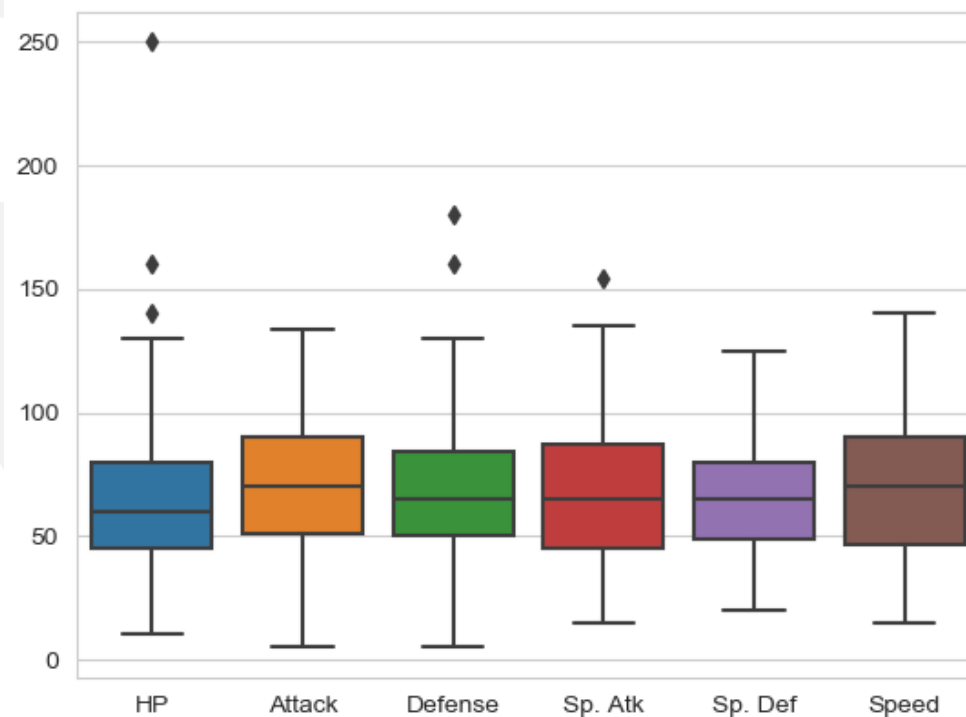
```
stats_df = df1.drop(['Total', 'Stage', 'Legendary'], axis=1)  
sns.boxplot(data=stats_df)
```



Seaborn's theme

- Seaborn has a number of themes you can use to alter the appearance of plots.
- For example, we can use “whitegrid” to add grid lines to our boxplot.

```
stats_df = df1.drop(['Total', 'Stage', 'Legendary'], axis=1)
sns.set_style('whitegrid')
sns.boxplot(data=stats_df)
```

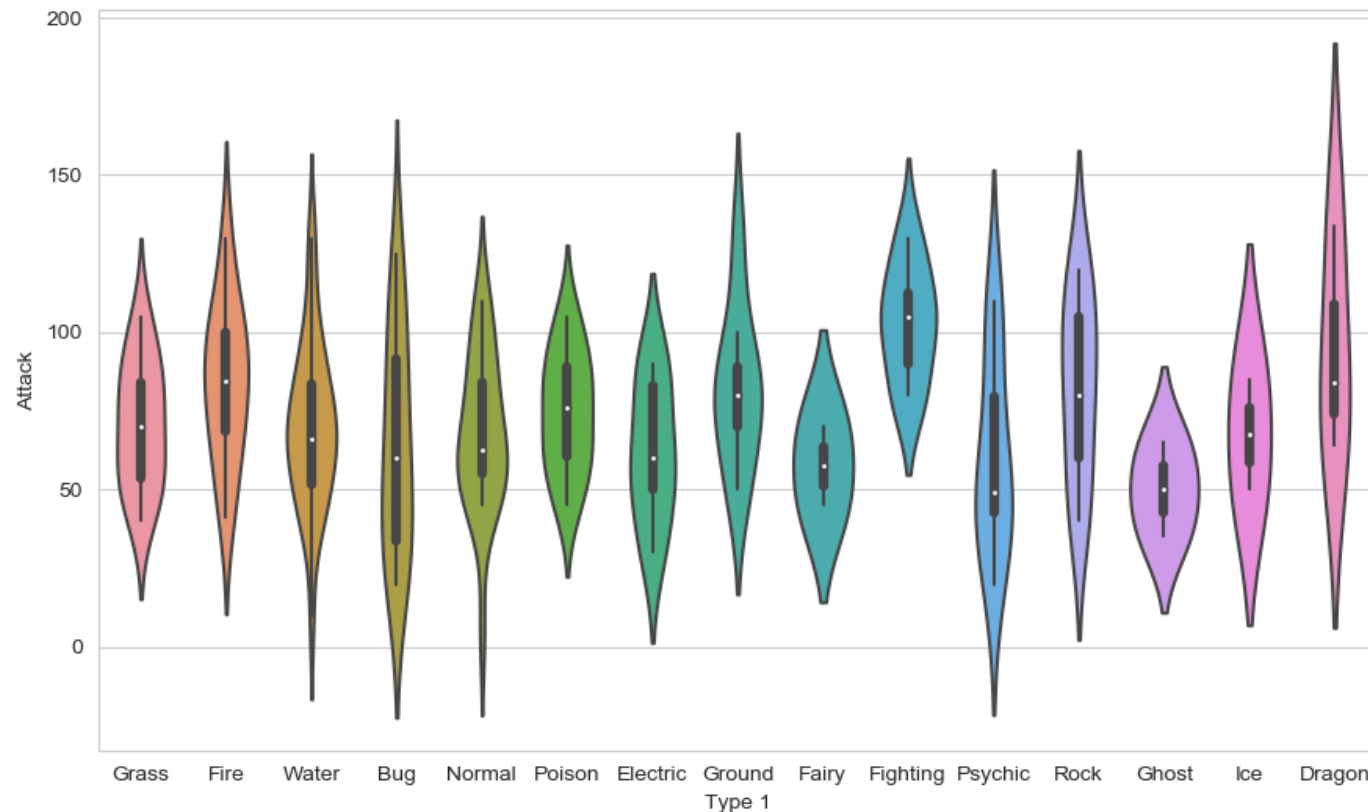


Violin plots

- Violin plots are useful alternatives to box plots.
- They show the distribution of a variable through the thickness of the violin.
- Here, we visualise the distribution of attack by Pokémon's primary type:



```
sns.violinplot(x='Type 1', y='Attack', data=df1)
```



- Dragon types tend to have higher Attack stats than Ghost types, but they also have greater variance. But there is something not right here....
- The colours!



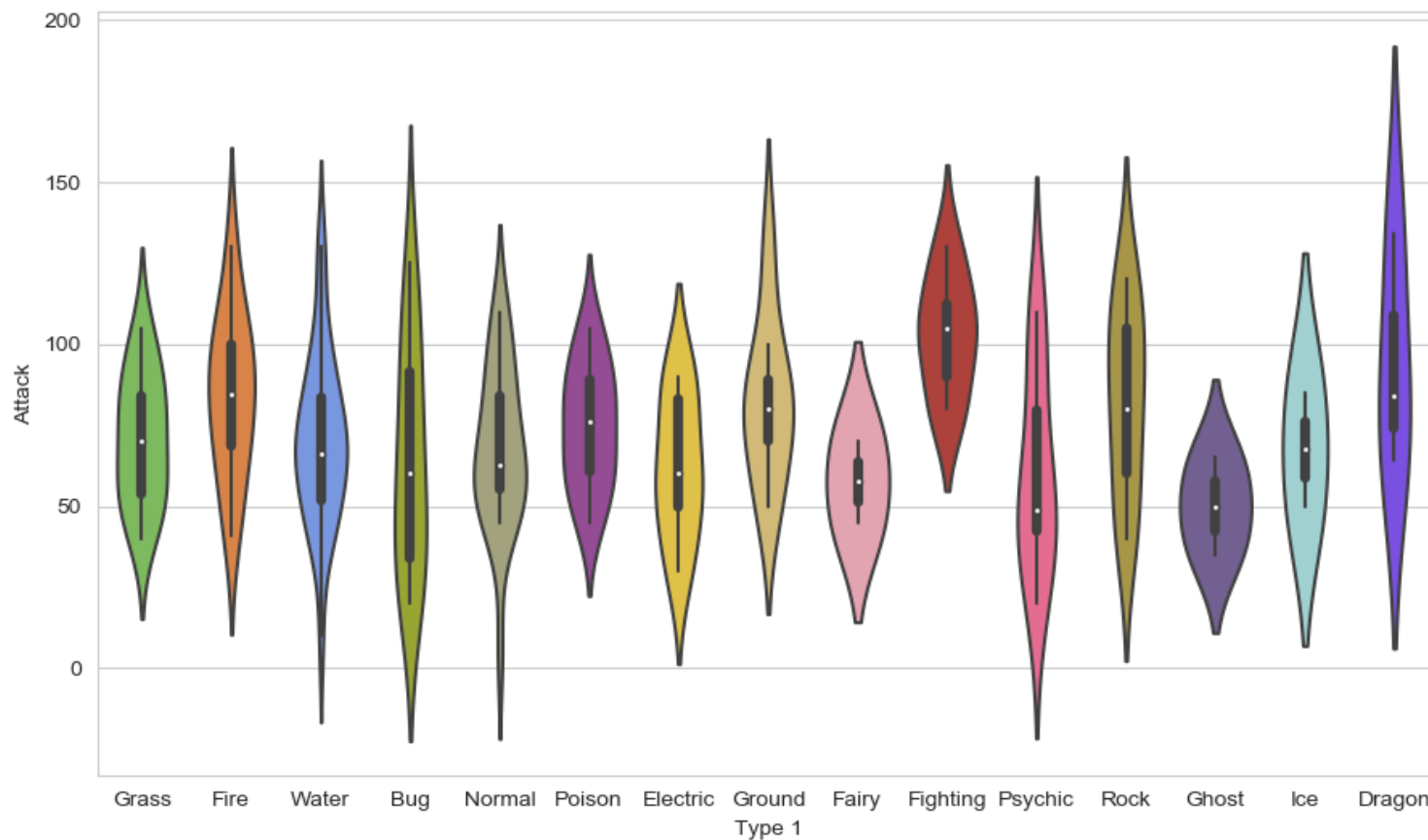
Seaborn's colour palettes

- Seaborn allows us to easily set custom colour palettes by providing it with an ordered list of colour hex values.
- We first create our colours list.

```
type_colors = ['#78C850', # Grass
               '#F08030', # Fire
               '#6890F0', # Water
               '#A8B820', # Bug
               '#A8A878', # Normal
               '#A040A0', # Poison
               '#F8D030', # Electric
               '#E0C068', # Ground
               '#EE99AC', # Fairy
               '#C03028', # Fighting
               '#F85888', # Psychic
               '#B8A038', # Rock
               '#705898', # Ghost
               '#98D8D8', # Ice
               '#7038F8', # Dragon
               ]
```

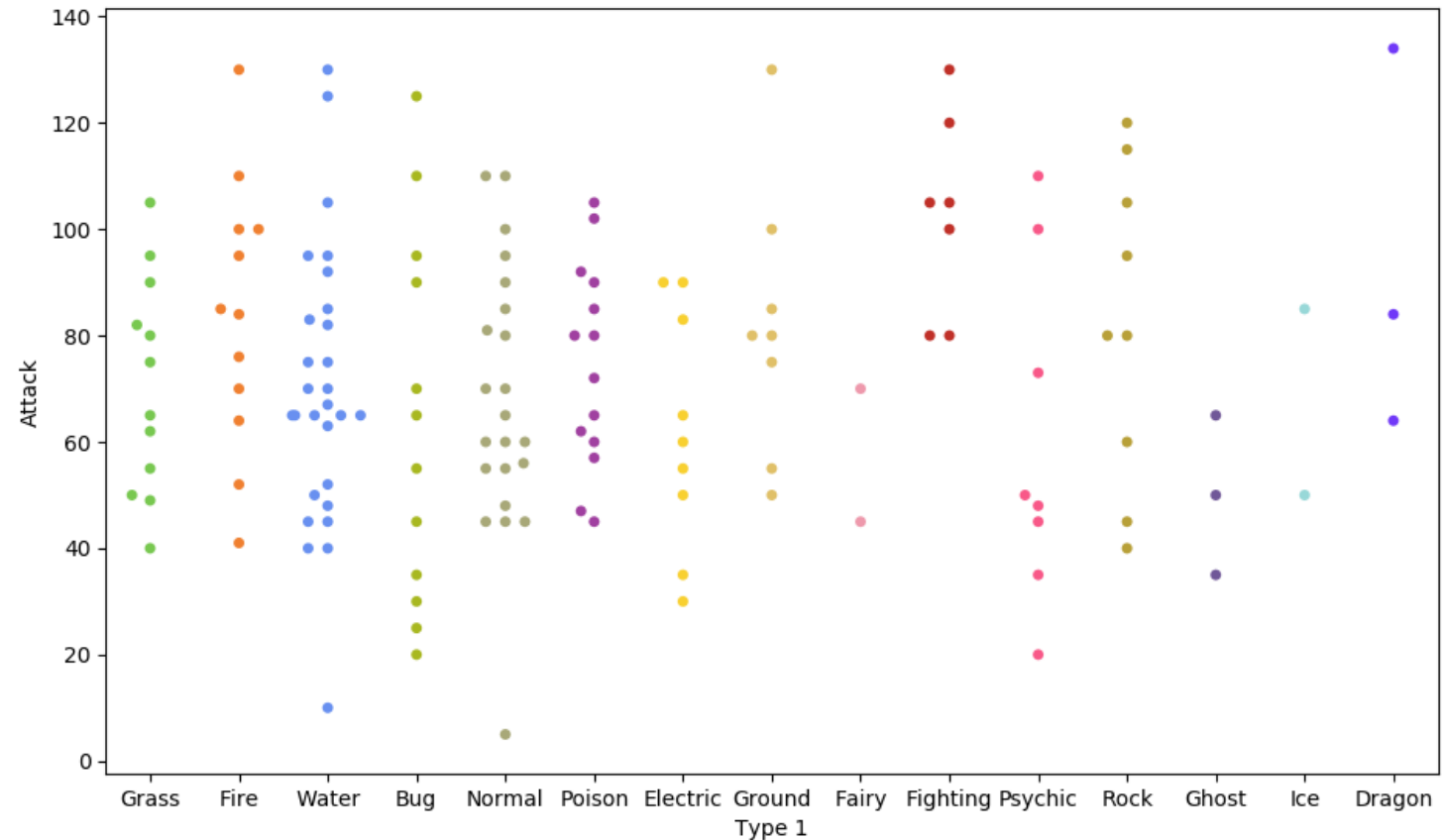
- Then we just use the `palette=` function and feed in our colours list.

```
sns.violinplot(x='Type 1', y='Attack', data=df1,  
               palette=type_colors)
```



- Because of the limited number of observations, we could also use a swarm plot.
- Here, each data point is an observation, but data points are grouped together by the variable listed on the x-axis.

```
sns.swarmplot(x='Type 1', y='Attack',  
              data=df1,  
              palette=type_colors)
```



Overlapping plots

- Both of these show similar information, so it might be useful to overlap them.

```
plt.figure(figsize=(10,6))  
sns.violinplot(x='Type 1',  
              y='Attack',  
              data=df1,  
              inner=None,  
              palette=type_colors)  
sns.swarmplot(x='Type 1',  
              y='Attack',  
              data=df1,  
              color='k',  
              alpha=0.7)  
plt.title('Attack by Type')
```

← Set size of print canvas.

← Remove bars from inside the violins

← Make bars black and slightly transparent

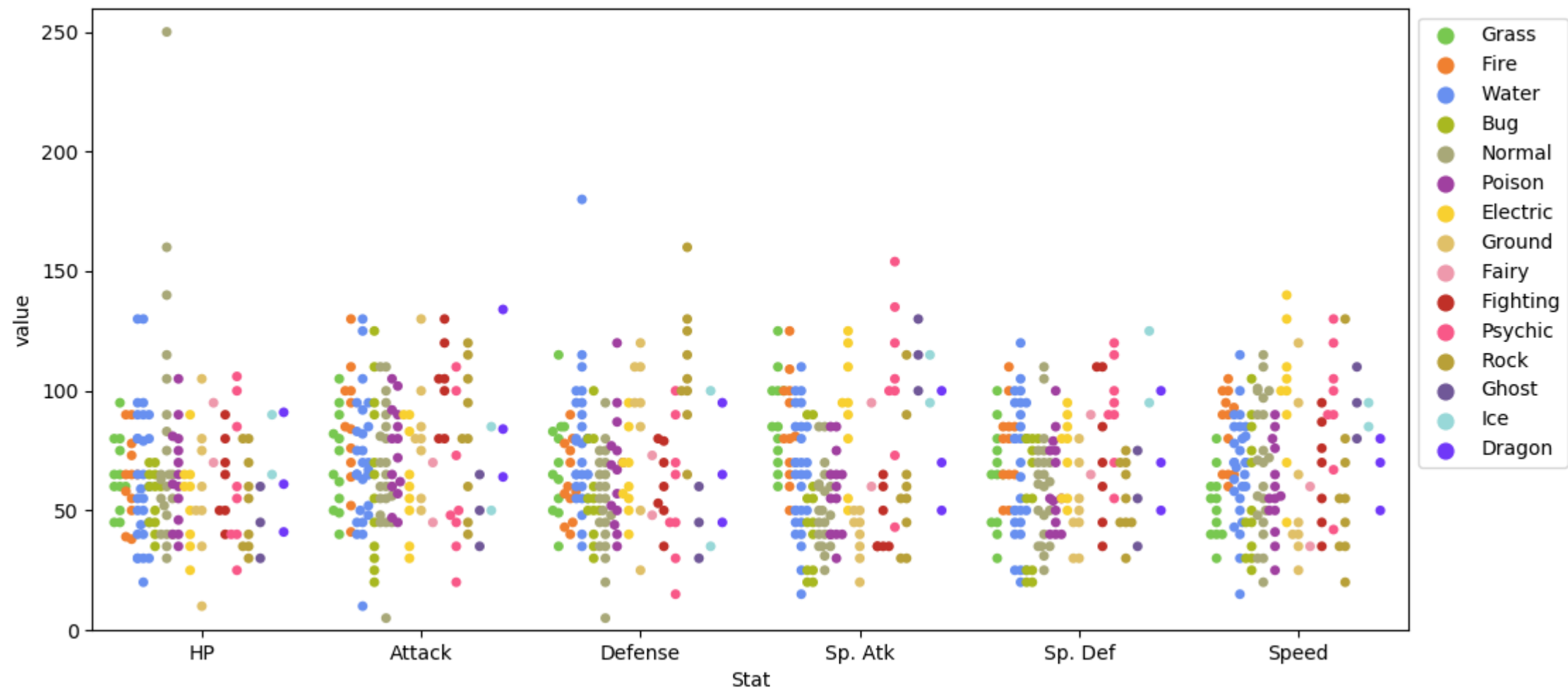
← Give the graph a title

Data wrangling with Pandas

- What if we wanted to create such a plot that included all of the other stats as well?
- In our current dataframe, all of the variables are in different columns:

```
print(df1.head())
```

#	Name	Type 1	Type 2	Total	...	Sp. Def	Speed	Stage	Legendary
1	Bulbasaur	Grass	Poison	318	...	65	45	1	False
2	Ivysaur	Grass	Poison	405	...	80	60	2	False
3	Venusaur	Grass	Poison	525	...	100	80	3	False
4	Charmander	Fire	NaN	309	...	50	65	1	False
5	Charmeleon	Fire	NaN	405	...	65	80	2	False



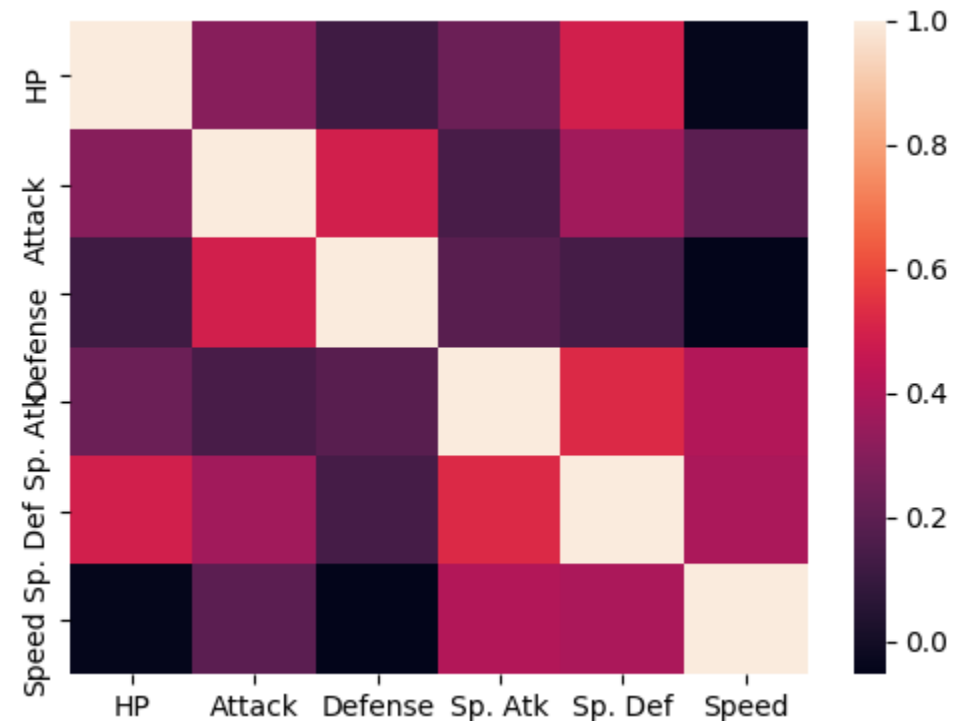
The usefulness of ECDFs

- It is often quite useful to plot the ECDF first as part of your workflow.
- It shows all the data and gives a complete picture as to how the data are distributed.

Heatmaps

- Useful for visualising matrix-like data.
- Here, we'll plot the correlation of the stats_df variables

```
corr = stats_df.corr()  
sns.heatmap(corr)
```

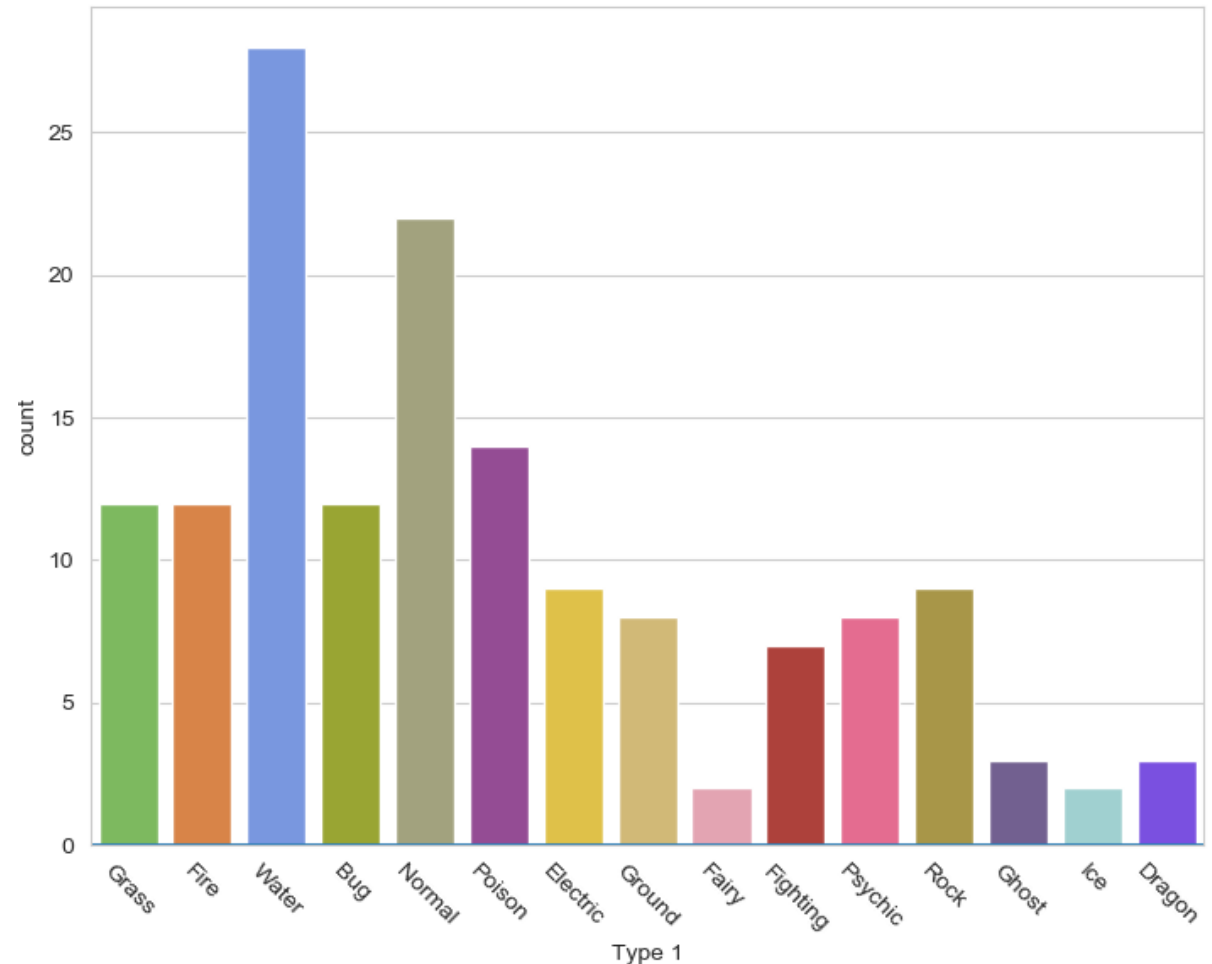


Bar plot

- Visualises the distributions of categorical variables.

```
sns.countplot(x='Type 1', data=df1,  
              palette=type_colors)  
plt.xticks(rotation=-45)
```

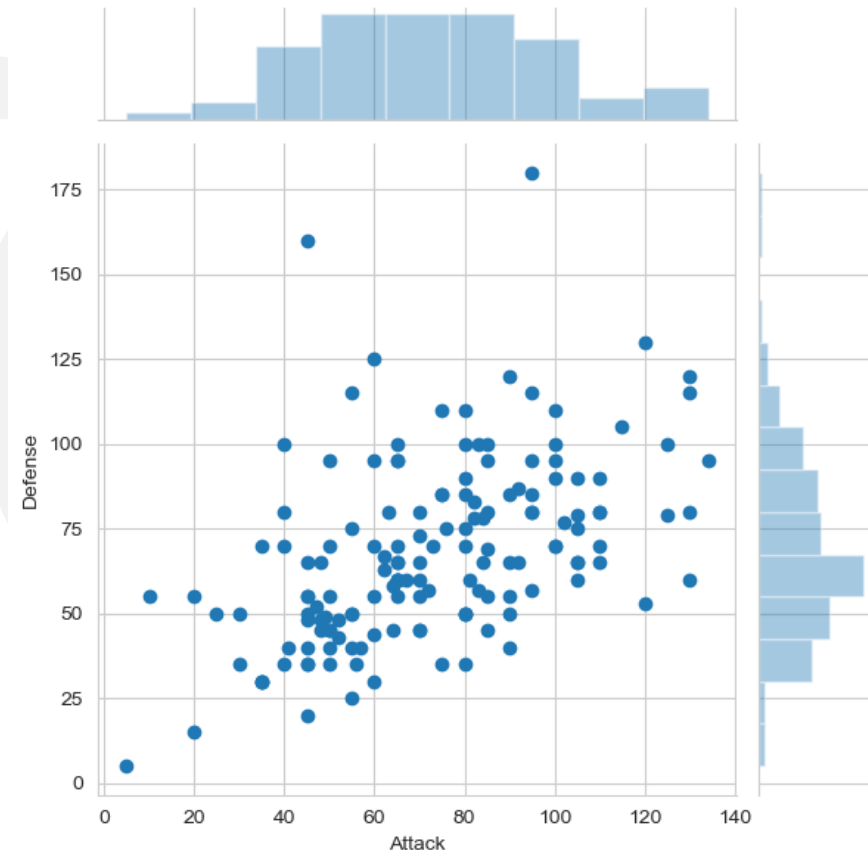
Rotates the x-ticks 45 degrees



Joint Distribution Plot

- Joint distribution plots combine information from scatter plots and histograms to give you detailed information for bi-variate distributions.

```
sns.jointplot(x='Attack',  
              y='Defense',  
              data=df1)
```



Thank you

A large, light gray watermark of a lion is positioned on the right side of the slide, facing left. The lion is depicted in a stylized, heraldic manner, with its front paw raised and its tail curved upwards.